Returns Automation Agent — Assignment Summary

Sourcing Return Automation Agent — browser automation for multi-item returns on Amazon and Flipkart.

1. What We Are Building

A script that reads pending product-return tasks from an Excel file, is meant to place each return on the correct e-commerce platform (Amazon or Flipkart), and writes the result back into the same Excel row. Each product in an order is tracked and resolved separately, not the order as a whole.

This is browser automation / RPA (Robotic Process Automation) — a script that follows a fixed, hardcoded sequence of steps. It is not an AI agent: there is no LLM, no API key, no model making judgment calls. Every decision (which platform, which flow type, pass or fail) is a hardcoded rule.

2. Why

Processing returns manually — open each order, pick a reason, confirm, note the outcome — is repetitive at any real order volume. The goal is to remove that repetitive manual step while keeping a human in the loop for anything the script can't safely resolve on its own (a blocked/verification page, an item out of its return window, an unexpected error).

3. How It Works

3.1 Processing loop

- Read every Excel row marked **To Do** / **Pending**.
- Group rows by Order ID + Platform, since one order can have several products (line items).
- For each order: open a browser session already logged in to that platform, and process each line item in that order one at a time.
- Write the result back to Excel immediately after each item — not batched at the end — so a crash mid-run doesn't lose completed work.
- Mark the whole order **Done** only once every line item in it has a final result.

3.2 Partial-success handling

If one item in an order fails (e.g. past its return window) the others still get processed. A single bad item never blocks or cancels the rest of the order — each line item is resolved independently and logged on its own row.

3.3 Login and credentials

Login happens once, manually, in a separate one-time setup script. The main agent never touches a login form or stores a password — it reuses a saved, already-authenticated browser session. This keeps credentials out of the code and out of any file that gets shared or committed to Git.

3.4 What happens on a blocked page

If a platform shows a CAPTCHA or identity-verification challenge, that line item is marked **Needs human review** and logged. The agent does not attempt to solve or bypass these challenges — that is intentionally left to a person.

4. Excel Structure

Column	Filled by	Purpose
Platform	Input	Amazon / Flipkart
OrderID	Input	Identifies the order
SKU	Input	Identifies the specific product in the order
ReturnWindow	Input	Used to check return eligibility
ReturnID	Agent	Captured from the platform after a successful return
ReturnStatus	Agent	Placed / Failed / Out of window
RefundAmount	Agent	As shown by the platform
TaskStatus	Agent	Done / Needs human review
Timestamp	Agent	When the row was processed

5. Current Status — What's Real vs. Placeholder

Built and working

- Excel read/write, per-line-item tracking, order grouping.
- Partial-success handling — one failed item never blocks the rest of the order.
- Retry-on-file-lock so a temporarily open Excel file doesn't crash the whole run.
- Login/session handling via a separate one-time script.

Not yet built

- The actual click-through steps on Amazon and Flipkart's return flow — currently placeholder code. No real return has been submitted on either platform.
- Real eligibility checking ("is this item out of its return window") — currently a stub.

This gap is the core remaining work: it requires inspecting a real, logged-in order page on each site and can't be completed without that live access.

6. Open Questions From the Spec

Amazon: the original spec listed Amazon as an example of both the "batch" return flow and the "sequential" return flow in the same document — these are contradictory. This build assumes sequential (one item at a time) for Amazon, pending confirmation against the real site.

Flipkart: the spec gave no flow-type detail for Flipkart at all — only login/test-order information. This build also assumes sequential for Flipkart, but that's an assumption carried over from Amazon, not something the spec actually stated. Needs the same confirmation.

7. How to Run

- **Install:** `pip install -r requirements.txt` then `playwright install chromium`
- **One-time login:** `python capture_login_session.py amazon` and the same for flipkart
- **Prepare Excel:** fill in Platform, OrderID, SKU, ReturnWindow, and set TaskStatus to **To Do** for each row to process
- **Close the Excel file** in Excel (Windows locks it while open — the script can't save results into a locked file)
- **Run:** `python main.py`